



Universidade Federal Rural do Semi-Árido – UFERSA

Campus Multidisciplinar de Pau dos Ferros – CMPF

Disciplina: Laboratório de Estrutura de Dados 2

Docente: Kennedy Reurison Lopes

Discentes: Carlos Ariel, Felype Kainan, John Kerry, Roger Leite

Sistema de Consulta Eficiente com Tabela Hash e Filtro de Bloom

Pau dos Ferros - 2026

1. Introdução

Este trabalho apresenta a implementação e avaliação experimental de um sistema de verificação de cadastro de usuários, desenvolvido inteiramente em linguagem C. O sistema combina duas estruturas de dados complementares: uma **Tabela Hash** com encadeamento externo, responsável pelo armazenamento e recuperação exata dos elementos, e um **Filtro de Bloom**, estrutura probabilística utilizada para acelerar consultas de existência, evitando acessos desnecessários à estrutura principal.

O objetivo é analisar o impacto do uso combinado dessas estruturas em termos de desempenho, consumo de memória e taxa de falsos positivos, respondendo às questões propostas pelo enunciado com base em dados experimentais coletados para conjuntos de 1.000, 10.000 e 100.000 registros.

2. Implementação

2.1 Tabela Hash

A Tabela Hash foi implementada com **encadeamento externo** (chaining). Cada posição da tabela aponta para uma lista encadeada de nós, permitindo que múltiplas chaves com o mesmo índice coexistam sem degradação brusca de desempenho.

Função hash utilizada — djb2 com XOR:

```
hash = 5381 | hash = (hash << 5 + hash) XOR c para cada caractere c
```

O multiplicador 33 (2^5+1) é eficiente computacionalmente e produz boa avalanche de bits para strings alfanuméricas curtas e longas.

Dimensionamento da tabela:

Parâmetro	Valor	Justificativa
Pior caso esperado (n)	100.000 elementos	Cenário máximo do experimento
Fator de carga alvo	0,70	Equilíbrio entre memória e colisões
Tamanho mínimo	142.857	$n / \text{fator de carga} = 100.000 / 0,70$
Tamanho adotado	142.909 (primo)	Próximo primo — reduz colisões por mod

Números primos como tamanho de tabela distribuem melhor as chaves pela operação módulo, reduzindo agrupamentos (clustering) que aumentariam o comprimento médio das cadeias.

2.2 Filtro de Bloom

O Filtro de Bloom foi implementado como um vetor de bits manipulado diretamente (bit a bit), com k funções hash virtuais geradas por **double hashing** a partir de duas funções base independentes (djb2 e sdbm):

```
 $g_i(x) = (h_1(x) + i \times h_2(x)) \bmod m \quad i = 0, 1, \dots, k-1$ 
```

Essa técnica, proposta por Kirsch e Mitzenmacher (2006), elimina a necessidade de implementar k funções completamente independentes, mantendo as garantias probabilísticas do Filtro de Bloom original.

Dimensionamento do filtro:

As fórmulas utilizadas, derivadas da teoria probabilística do Filtro de Bloom, são:

$$m = -(n \times \ln p) / (\ln 2)^2 \quad k = (m / n) \times \ln 2$$

Parâmetro	Valor	Justificativa
n (elementos esperados)	100.000	Pior cenário do experimento
p (taxa de FP alvo)	1% (0,01)	Aceitável para o domínio
m (tamanho do vetor)	958.507 bits (~117 KB)	$m = -(100000 \times \ln 0,01) / (\ln 2)^2 \approx 958.506$
k (funções hash)	7	$k = (958507/100000) \times \ln 2 \approx 6,64 \rightarrow 7$
Técnica multi-hash	Double hashing	Kirsch & Mitzenmacher (2006)

3. Funcionalidades do Sistema

O sistema oferece um menu interativo com as seguintes operações:

Opção	Funcionalidade	Descrição
1	Inserir usuário (RF01)	Cadastra novo ID na Hash e no Bloom
2	Consultar usuário (RF02)	Fluxo Bloom → Hash com rastreio de FP
3	Estatísticas (RF03)	Exibe métricas completas do sistema
4	Carregar arquivo (RF04)	Insere em lote a partir de arquivo .txt
5	Experimentos	Gera arquivos e executa os 3 cenários de teste

O fluxo de consulta (RF02) segue obrigatoriamente a sequência: (1) consulta o Filtro de Bloom; (2) se o resultado for 'definitivamente não existe', retorna imediatamente sem acessar a Hash; (3) se for 'possivelmente existe', consulta a Tabela Hash; (4) informa o resultado final, distinguindo verdadeiros positivos de falsos positivos.

4. Experimentos e Resultados

Os experimentos foram executados com conjuntos de 1.000, 10.000 e 100.000 registros gerados aleatoriamente no formato [8 letras][3 dígitos] (ex.: islaifda122). Cada cenário realiza um conjunto de consultas misto: 50% de IDs presentes na base (verdadeiros positivos esperados) e 50% de IDs ausentes (onde o Bloom demonstra sua vantagem ao evitar o acesso à Hash).

N	Tempo s/ Bloom	Tempo c/ Bloom	Consultas Evitadas	Falsos Positivos	Taxa FP
1.000	0,03 ms	0,08 ms	1.000 / 2.000 (50%)	0	0,0000%
10.000	0,65 ms	0,92 ms	10.000 / 20.000 (50%)	0	0,0000%
100.000	10,46 ms	15,66 ms	99.078 / 200.000 (49,5%)	922	0,9220%

Tabela 1 — Resultados dos experimentos

4.1 Análise dos Resultados

Para conjuntos pequenos (1k e 10k), o tempo com Bloom foi ligeiramente maior que sem Bloom. Isso ocorre porque o overhead do cálculo das 7 funções hash do filtro supera o ganho obtido ao evitar a consulta na tabela, já que a própria tabela Hash é muito eficiente para poucos elementos. Entretanto, para 100.000 registros, o Bloom evitou aproximadamente 49,5% das consultas à Hash, demonstrando sua utilidade em escala.

A taxa de falsos positivos observada (0,9220%) ficou muito próxima do valor teórico configurado (1%), validando o dimensionamento adotado com $m = 958.507$ bits e $k = 7$ funções hash.

5. Respostas às Questões do Enunciado

1. O Filtro de Bloom reduziu consultas na Tabela Hash?

Sim. No cenário de 100.000 registros, 99.078 de 200.000 consultas (49,5%) foram descartadas pelo Bloom antes de chegar à Hash. Para os conjuntos menores, o Bloom também descartou 50% das consultas, porém o overhead computacional do filtro tornou o tempo total ligeiramente maior nesses casos.

2. Como o tamanho do vetor impactou os resultados?

O vetor de 958.507 bits foi dimensionado para suportar até 100.000 elementos com taxa de falso positivo de 1%. O experimento confirmou essa estimativa (0,9220% observado). Um vetor menor elevaria a taxa de FP; um vetor maior reduziria os FP mas aumentaria o consumo de memória.

3. O número de funções hash alterou a precisão?

Sim. O número ótimo de funções hash $k = 7$ foi calculado pela fórmula $k = (m/n) * \ln 2$. Valores de k abaixo do ótimo reduzem a precisão (mais FP); valores acima aumentam o custo computacional sem ganho proporcional. As 7 funções adotadas produziram taxa de FP próxima ao teórico de 1%, confirmando a escolha correta.

4. Em qual cenário o Bloom foi mais vantajoso?

O Bloom foi mais vantajoso no cenário de 100.000 registros, onde evitou quase metade das consultas à Hash. Em aplicações reais com milhões de registros, especialmente quando a Hash está em disco ou memória lenta, o ganho seria ainda mais expressivo.

6. Conclusão

O sistema implementado demonstrou o funcionamento correto e eficiente da combinação entre Tabela Hash e Filtro de Bloom para verificação de cadastro de usuários. As decisões de projeto — tamanho primo da tabela (142.909), vetor de bits com 958.507 posições e 7 funções hash — foram tomadas com base em fórmulas teóricas e validadas experimentalmente.

Os experimentos confirmaram que o Filtro de Bloom é mais vantajoso em grandes volumes de dados. A taxa de falsos positivos observada (0,92%) ficou dentro do alvo estabelecido (1%), validando o dimensionamento adotado.

7. Instruções de Compilação e Execução

Pré-requisitos: GCC e biblioteca matemática (libm).

Compilação (Linux/macOS):

```
gcc -O2 -Wall -o sistema src/hash.c src/bloom.c src/main.c -lm
```

Compilação (Windows com MinGW):

```
gcc -O2 -Wall -o sistema src/hash.c src/bloom.c src/main.c
```

Execução:

```
./sistema
```

Selecionar a opção **5** no menu para executar os experimentos automatizados com geração dos arquivos de teste.